D20A / 52

EXPERIMENT 10

1. Aim

To implement a Hyperledger Fabric blockchain network and perform asset management operations using chaincode, including creating, reading, updating and transferring ownership of digital assets.

2. Theory

Hyperledger Fabric is an open-source, permissioned blockchain framework developed by the Linux Foundation. It is designed for enterprise applications where multiple organizations need to securely share information and perform transactions.

Unlike public blockchains such as Bitcoin and Ethereum, Hyperledger Fabric allows only authorized participants to access the network. It provides privacy, security, scalability and controlled access to blockchain transactions.

Hyperledger Fabric uses a modular architecture consisting of organizations, peer nodes, ordering services, channels and smart contracts called chaincode.

Components of Hyperledger Fabric

1. **Organizations:** Represent different participants in the blockchain network. In this experiment, two organizations, Org1 and Org2, are created.
2. **Peer Nodes:** Maintain the blockchain ledger and execute chaincode to validate and process transactions.
3. **Ordering Service:** Organizes transactions into blocks and distributes them to the participating peers.
4. **Channel:** Provides a private communication mechanism between organizations. A channel named `mychannel` is created for this experiment.
5. **Chaincode:** Represents the smart contract containing business logic. It defines how assets are created, retrieved, modified and transferred.
6. **Ledger:** Stores the blockchain transaction history and the current state of assets.
7. **Certificate Authority (CA):** Provides digital identities and certificates to network participants for authentication.

Asset Management Using Chaincode

Asset management is the process of creating, storing, tracking and transferring ownership of digital assets on a blockchain network.

In this experiment, the official Asset Transfer Basic chaincode provided by Hyperledger Fabric is used. It is written in the Go programming language.

The following operations are performed:

- **CreateAsset:** Creates a new asset and stores its information on the ledger.
- **ReadAsset:** Retrieves the details of an existing asset.
- **UpdateAsset:** Modifies the properties of an existing asset.
- **TransferAsset:** Transfers ownership of an asset to another participant.
- **GetAllAssets:** Retrieves all the assets stored in the ledger.
- **InitLedger:** Initializes the ledger with sample assets.

The experiment demonstrates how Hyperledger Fabric provides a secure and shared environment for managing digital assets across multiple organizations.

3. Procedure / Steps of Implementation

Step 1: Configure the Environment

```
PS C:\Users\Diya> docker --version
Docker version 28.0.4, build b8034c0
PS C:\Users\Diya> docker compose version
Docker Compose version v2.34.0-desktop.1
PS C:\Users\Diya> wsl -d Ubuntu
diya@LAPTOP-TJ780EQ2:/mnt/c/Users/Diya$ docker --version
Docker version 28.0.4, build b8034c0
diya@LAPTOP-TJ780EQ2:/mnt/c/Users/Diya$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Step 2: Install Hyperledger Fabric

```
diya@LAPTOP-TJ780EQ2:/mnt/c/Users/Diya$ cd ~
diya@LAPTOP-TJ780EQ2:~$ mkdir -p fabric-project
diya@LAPTOP-TJ780EQ2:~$ cd ~/fabric-project
diya@LAPTOP-TJ780EQ2:~/fabric-project$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
diya@LAPTOP-TJ780EQ2:~/fabric-project$ chmod +x install-fabric.sh
diya@LAPTOP-TJ780EQ2:~/fabric-project$ ls -l install-fabric.sh
-rwxr-xr-x 1 diya diya 11135 Nov  5 12:09 install-fabric.sh
diya@LAPTOP-TJ780EQ2:~/fabric-project$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

===> Cloning hyperledger/fabric-samples repo
Cloning into 'fabric-samples'...
remote: Enumerating objects: 16063, done.
remote: Counting objects: 100% (3470/3470), done.
remote: Compressing objects: 100% (641/641), done.
remote: Total 16063 (delta 3045), reused 2829 (delta 2829), pack-reused 12593 (from 3)
Receiving objects: 100% (16063/16063), 24.08 MiB | 3.08 MiB/s, done.
Resolving deltas: 100% (9276/9276), done.
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.
```

```
===> List out hyperledger images
ghcr.io/hyperledger/fabric-peer      2.5.16          a3835b028353   4 months ago   155MB
hyperledger/fabric-peer              2.5             a3835b028353   4 months ago   155MB
hyperledger/fabric-peer              2.5.16          a3835b028353   4 months ago   155MB
hyperledger/fabric-peer              latest          a3835b028353   4 months ago   155MB
hyperledger/fabric-orderer           2.5             de354d43b52b   4 months ago   121MB
hyperledger/fabric-orderer           2.5.16          de354d43b52b   4 months ago   121MB
hyperledger/fabric-orderer           latest          de354d43b52b   4 months ago   121MB
ghcr.io/hyperledger/fabric-orderer   2.5.16          de354d43b52b   4 months ago   121MB
hyperledger/fabric-ccenv             2.5             c011802bedda   4 months ago   675MB
hyperledger/fabric-ccenv             2.5.16          c011802bedda   4 months ago   675MB
hyperledger/fabric-ccenv             latest          c011802bedda   4 months ago   675MB
ghcr.io/hyperledger/fabric-ccenv     2.5.16          c011802bedda   4 months ago   675MB
ghcr.io/hyperledger/fabric-baseos    2.5.16          94ad0ab8a77f   4 months ago   160MB
hyperledger/fabric-baseos            2.5             94ad0ab8a77f   4 months ago   160MB
hyperledger/fabric-baseos            2.5.16          94ad0ab8a77f   4 months ago   160MB
hyperledger/fabric-baseos            latest          94ad0ab8a77f   4 months ago   160MB
hyperledger/fabric-ca                1.5             0548296e36d4   9 months ago   244MB
hyperledger/fabric-ca                1.5.17          0548296e36d4   9 months ago   244MB
hyperledger/fabric-ca                latest          0548296e36d4   9 months ago   244MB
ghcr.io/hyperledger/fabric-ca        1.5.17          0548296e36d4   9 months ago   244MB
```

Step 3: Start the Fabric Network

```
- CC_SRC_LANGUAGE: go
- CC_VERSION: 1.0
Vendoring Go dependencies at ../asset-transfer-basic/chaincode-go
~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go ~/fabric-project/fabric-samples/test-network
~/fabric-project/fabric-samples/test-network
Finished vendoring Go dependencies
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-go --lang golang --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:d0b27e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-11-05 12:37:29.302 UTC 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed remotely: response:<status:200 payload:"\nJbasic_1.0:d0b2
7e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa\022\tbasic_1.0" >
2026-11-05 12:37:29.303 UTC 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identifier: basic_1.0:d0b27e1af09f3039dabac8
2678c64e8807311f9405008efb867ba8895dd643aa
Chaincode is installed on peer0.org1
Install chaincode on peer0.org2...
Using organization 2
+ peer lifecycle chaincode queryinstalled --output json
+ grep '^basic_1.0:d0b27e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa$'
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-11-05 12:38:01.608 UTC 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed remotely: response:<status:200 payload:"\nJbasic_1.0:d0b2
7e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa\022\tbasic_1.0" >
2026-11-05 12:38:01.608 UTC 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identifier: basic_1.0:d0b27e1af09f3039dabac8
2678c64e8807311f9405008efb867ba8895dd643aa
Chaincode is installed on peer0.org2
++ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
++ sed -n '/Version:/{s/.*Sequence: //; s/, Endorsement Plugin:.*$//; p;}'
Error: query failed with status: 404 - namespace basic is not defined
+ COMMITTED_CC_SEQUENCE=
+ res=0
```

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ cd ~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go$
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go$ go mod vendor
go: downloading go1.25.0 (linux/amd64)
go: downloading github.com/hyperledger/fabric-chaincode-go/v2 v2.0.0
go: downloading github.com/hyperledger/fabric-contract-api-go/v2 v2.2.0
go: downloading github.com/hyperledger/fabric-protos-go-apiv2 v0.3.4
go: downloading google.golang.org/protobuf v1.36.11
go: downloading github.com/stretchr/testify v1.10.0
go: downloading github.com/pmezard/go-difflib v1.0.0
go: downloading github.com/davecgh/go-spew v1.1.1
go: downloading gopkg.in/yaml.v3 v3.0.1
go: downloading google.golang.org/grpc v1.83.2
go: downloading github.com/xeipuuv/gojsonschema v1.2.0
go: downloading github.com/go-openapi/spec v0.21.0
go: downloading github.com/xeipuuv/gojsonreference v0.0.0-20180127040603-bd5ef7bd5415
go: downloading github.com/go-openapi/swag v0.23.0
go: downloading github.com/go-openapi/jsonpointer v0.21.0
go: downloading github.com/go-openapi/jsonreference v0.21.0
go: downloading github.com/xeipuuv/gojsonpointer v0.0.0-20190905194746-02993c407bfb
go: downloading github.com/mailru/easyjson v0.7.7
go: downloading golang.org/x/sys v0.47.0
go: downloading google.golang.org/genproto v0.0.0-20230410155749-daa745c078e1
go: downloading golang.org/x/net v0.58.0
go: downloading github.com/josharian/intern v1.0.0
go: downloading golang.org/x/text v0.41.0
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go$ cd ~/fabric-project/fabric-samples/test-network
```

Step 4: Deploy Asset Management Chaincode

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-go
- CC_SRC_LANGUAGE: go
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-go
- CC_SRC_LANGUAGE: go
- CC_VERSION: 1.0
Vendoring Go dependencies at ../asset-transfer-basic/chaincode-go
~/fabric-project/fabric-samples/asset-transfer-basic/chaincode-go ~/fabric-project/fabric-samples/test-network
~/fabric-project/fabric-samples/test-network
Finished vendoring Go dependencies
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-go --lang golang --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:d0b27e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-11-05 12:37:29.302 UTC 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed remotely: response:<status:200 payload:"\nJbasic_1.0:d0b2
7e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa\022\tbasic_1.0" >
2026-11-05 12:37:29.303 UTC 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identifier: basic_1.0:d0b27e1af09f3039dabac8
```

Step 5: Configure Organization 1

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID=Org1MSP

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051
```

Step 6: Configure Network Certificates

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ export ORDERER_CA=${PWD}/organizations/ordererOrganizations/example
reaamies/tscos/example.comms/example.omcer-cert.pem

export PEER0_ORG1_CA=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export PEER0_ORG2_CA=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

Step 7: Initialize the Ledger

The ledger was initialized with sample assets using the `InitLedger` function.

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PEER0_ORG1_CA" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PEER0_ORG2_CA" \
-c '{"functior":"InitLedger","Argst:[]}'
2026-11-05 12:51:05.109 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Step 8: Retrieve All Assets

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode query \
-C mychannel -n basic \
-c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5},{"
{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10},
},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size"
:15}]
```

Step 9: Create a New Asset

Created a new asset with ID `asset101`.

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PEER0_ORG1_CA" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PEER0_ORG2_CA" \
-c '{"function":"CreateAsset","Args":["asset101","purple","10","Alice","75000"]}'
```

Step 10: Read Asset Details

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode query \
-C mychannel -n basic \
-c '{"function":"ReadAsset","Args":["asset101"]}'
{"AppraisedValue":75000,"Color":"purple","ID":"asset101","Owner":"Alice","Size":10}
```

Step 11: Update Asset Information

The asset details were modified by changing its color, size, owner and appraised value.

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PEER0_ORG1_CA" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PEER0_ORG2_CA" \
-c '{"function":"UpdateAsset","Args":["asset101","blue","12","Bob","85000"]}'
```

Step 12: Transfer Asset Ownership

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PEER0_ORG1_CA" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PEER0_ORG2_CA" \
-c '{"function"":"TransferAsset","Args":[:"asset101","Charlie"]}'
2026-11-05 12:51:47.100 UTC 0001  INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Bob"
```

Step 13: Verify the Final Asset

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode query \
-C mychannel -n basic \
-c '{"function":"ReadAsset","Args":["asset101"]}'
{"AppraisedValue":85000,"Color":"blue","ID":"asset101","Owner":"Charlie","Size":12}
```

Step 14: Verify Through Organization 2

The peer environment was switched to Org2.

The asset was queried again using Org2 to verify that the updated asset information was available to both organizations.

```
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ export CORE_PEER_LOCALMSPID=Org2MSP

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp

export CORE_PEER_ADDRESS=localhost:9051
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$ peer chaincode query \
-C mychannel -n basic \
-c '{"function":"ReadAsset","Args":["asset101"]}'
{"AppraisedValue":85000,"Color":"blue","ID":"asset101","Owner":"Charlie","Size":12}
diya@LAPTOP-TJ780EQ2:~/fabric-project/fabric-samples/test-network$
```

4. Conclusion

Thus, the Hyperledger Fabric network was successfully implemented using Docker and WSL2. A channel named `mychannel` was created with two organizations, and Asset Management Chaincode was deployed successfully. Various operations such as asset creation, reading, updating and ownership transfer were performed.